INF2009 Project Overview

Asst Prof. Tan Zheng Hui Ernest

INF2009 – Edge Computing and Analytics

What Makes a Problem Suitable for Edge Computing

Project Requirements (compulsory)

- Real-time constraints (sub-100 ms loops)
- Heavy sensor input (video/audio/IMU)
- Privacy boundaries
- Intermittent or unreliable connectivity
- Situations where actuation decisions must not depend on cloud

Teams are to create a one-slider illustrating a coherent system architecture diagram containing:

- Sensors
- Compute nodes (edge + optional cloud)
- Interconnecting protocols (I^2C, SPI, UART, MQTT, REST)
- Data flow direction
- Where decisions are made and what happens if each component fails

The one-slider must explicitly capture architecture with failure handling and offline behaviour

- Do not include pretty boxes with **no operational clarity**.
- This enforces evidence-based engineering.

Model/Component Interaction Latency Budget Table

Stage	Latency (ms)	Notes
Capture	10	Camera driver overhead
Preprocess	4	Resize/normalize
Model A	22	Pose estimation
Model B	9	Gesture classifier
Fusion	3	Rules + smoothing
Actuation	4	Motor control

Teams are to prepare a Model/Component Interaction Latency Budget Table

- Latency breakdown per component
- Pipeline latency budget

This table forces you to think quantitatively.

Resource Budget Summary (CPU, RAM, Storage)

Teams often have no idea that:

- Their RAM will run out
- Two models can't run at once
- Storage logs will fill up
- CPU saturation causes frame drops

Teams are to add a requirement for:

- Expected CPU usage
- Expected GPU/NPU usage
- RAM allocation per model
- Expected max storage footprint

This also reinforces the importance of “PC capping.”

Week 8 “Design & Justification” Submission

A Clear Checklist for the Week 8 “Design & Justification” Submission

- Problem definition with real-time latency and processing requirements (latency/bandwidth/privacy/power)
- System architecture diagram
- Problem decomposition + model mapping
- Model selection justification table
- Scheduling plan & timeline diagram
- Platform choice & trade-off matrix
- PC capping strategy (if applicable)
- Measurement plan (even if preliminary)

Common architectural mistakes

- Doing inference in cloud
- Trying to run YOLOv8-Large on RPi
- Using one giant model for everything
- Having no fallback behaviour

Project Rationale

- Module focus is on building edge-first solutions that meet real-world constraints (latency, bandwidth, privacy, power).
- Deliverables and rubrics overview are a good indication of what “good” looks like.
    - Here, **PC-only/cloud-heavy ideas won’t cut it in this module**.

Why Edge (Not Cloud)

Make the case in the problem space, not just the tech space:

- **Latency and Control Loops**
  - Sub 100 ms decisions for actuation, safety interlocks, or machine vision quality checks; cloud RTT/jitter makes this unreliable.
- **Communications and Processing Efficiency**
  - Continuous raw sensor streams (video, IMU, audio) are expensive to uplink and pre filter/aggregate at the edge.
- **Privacy Boundaries**
  - Keep raw patient/worker data and video local, uplink only features/metadata.
- **Intermittent Connectivity**
  - Field, factory, or campus dead zones require offline-first operation with graceful cloud fallback.
- **Energy Consumption and Cost**
  - Edge devices can be optimized for watts-per-inference vs. paying for persistent cloud compute & egress.

Students must argue from scenario constraints (latency budget, bandwidth, privacy, availability, power) rather than from preference.

“Edge vs Cloud” Justification: What Good Evidence Looks Like

For each project, measurable justifications are required:

- **Latency budget:**
  - Show an end-to-end timing diagram
  - For example, instrument timestamps at capture → preprocess → inference → decision → actuation.
- **Network dependence**
  - Demonstrate degraded/zero network using emulator (e.g., packet loss 10–30%, 200–500 ms added RTT) and show continued operation.
- **Data minimization**
  - Compare uplink volume for raw vs. edge-processed payloads (e.g., frames/sec vs. events/sec).
- **Privacy boundary**
  - Identify private information in raw streams and show how on device feature extraction avoids transmitting sensitive data.
- **Energy profile**
  - Measure idle/active power and estimate battery/runtime or thermal constraints if relevant.

This will be required in the Design & Justification submission (Week 8), pick which ones with justification.

Platform Selection: How to Pick (and Defend) Your Edge Device

Decision factors which students must justify each:

- **Compute & accelerators:**
  - CPU/GPU/NPU/TPU vs. model size/throughput; INT8 availability; real-time threads.
- **Memory & storage:**
  - Model + buffers + OS headroom; sustained write for logs.
- **I/O & interfaces:**
  - Cameras (MIPI/USB), sensors (SPI/I^2C/CAN), comms (Wi-Fi/BT/LTE), GPIO for actuators.
- **Power & thermals:**
  - Idle/peak watts, battery feasibility, thermal throttling behavior.
- **Software stack:**
  - OS/RTOS, driver support, SDKs (e.g., TensorRT, OpenVINO), container support.
- **Form factor & ruggedness:**
  - Mounting, temperature range, ingress protection.
- **Cost & availability:**
  - Lead time, spares for demo day.
- **Community & documentation:**
  - Longevity and examples reduce integration risk.

Tie the above to the "Design & Justification" checkpoint

- **No device choice = no credible edge story.**

Platform Selection: How to Pick (and Defend) Your Edge Device

Example Platform "Shortlist" (illustrative, pick what matches your lab stock):

- **General-purpose edge AI:**
  - NVIDIA Jetson (Nano/Orin), Google Coral, Intel NUC/Arc (with containers).
- **Constrained embedded:**
  - Raspberry Pi 4/5 with HATs, ESP32/STM32 + external sensors, Arduino Portenta H7.
- **Industrialized:**
  - Compute modules with carrier boards, PoE cameras, fanless x86 boxes.
- **How to defend a choice:**
  - Map device specs to your **measured** latency/throughput targets, I/O needs, and privacy boundaries; include a one-page trade-off table.

What to Do If the Ideal Platform Isn't Available

Onus is on the team to *cap* a PC to emulate the intended embedded envelope.

- Require them to document the caps used in demos and in their repo.
- **CPU & cores**
  - Pin to N cores / throttle frequency (e.g., limit to 1–2 cores, cap at 1.2 GHz).
  - Run containers/VMs with explicit quotas (--cpus, --cpuset-cpus).
- **Memory**
  - Constrain to target RAM (e.g., 512 MB–2 GB) via container/VM memory limits; prove no swap thrash.
- **I/O & storage**
  - Limit disk throughput/IOPS to SD-card-like speeds; pre-allocate small log partitions.
- **Network**
  - Emulate lossy/slow links (e.g., add 200 ms RTT, 1–5 Mbps ceilings, 1–5% loss) to show graceful degradation.
- **Power/Thermals**
  - Run on a power budget (USB power meter); demonstrate behavior under thermal throttle (reduced frame rate, back-off strategies).
- **Sensors**
  - Rate-limit capture (e.g., 5–10 FPS) and enforce on-device pre-processing before any uplink.

Evidence Pack Students Must Submit (Design & Justification)

Scenario-driven non-functional requirements

- Latency, privacy, bandwidth, availability, power) with numeric targets.

Measurement traces

- Timing logs, throughput graphs, power samples).

Before/after

- Cloud-dependent vs. edge-first behavior under network and CPU/memory caps.

Platform trade-off matrix and final rationale

Risk register and mitigations

- Supply chain, drivers, thermal, out-of-stock.

Common Pitfalls (and How to Avoid Them)

- **"We used a PC because it's convenient."** → **Not acceptable**: **cap** it and show edge equivalence.
- **Unmeasured claims** ("low latency," "privacy-preserving"). → Measure & visualize.
- **Device-driven projects** ("we had a Jetson so we used it"). → **Start from constraints**; back into the platform.
- **Cloud lock-in for core loop** (e.g., inference in cloud). → Keep the **decision loop** on-device; use cloud for fleet mgmt/updates/analytics.

Problem Decomposition, Model Selection, Scheduling

Designing the System: Break the Problem Into Components

- To build a credible edge-first solution, students must decompose the problem instead of treating it as one giant model or pipeline.
- Break the scenario into independent, observable sub-problems, each backed by a model or algorithm:

Examples (for human-centric projects):

- Human detection – Bounding box detection
- Arm/hand pose estimation – Keypoint detection for arm angles, wrist motion
- Face analytics – Head pose, eye gaze, blink detection
- Gesture classification – Small classifier running on cropped regions
- Activity recognition – IMU + vision fusion model

Why decomposing matters

- Each component has different compute needs (CPU? GPU? NPU?).
- Smaller models = lower latency, easier to fit in memory.
- Components can be scheduled independently, enabling real-time performance.
- Easier to swap or upgrade individual models.

Choosing the Right Models (Not “Whatever Works on PC”)

SINGAPORE INSTITUTE OF TECHNOLOGY

Students must justify why each model chosen is appropriate for the edge constraints of their selected platform.

Model Selection Factors

- Model size: For example, MobileNet, BlazePose, YOLO-Nano vs YOLOv8-Large
- Latency budget: Which model can run at ≥ X FPS on the target device?
- Accuracy vs. cost trade-off: Sometimes a slightly less accurate but faster model is correct for the edge.
- Input resolution: Lower resolutions may still meet functional accuracy thresholds.
- Quantization: INT8, FP16 for edge NPUs (Jetson, Coral)
- Pipeline dependency: If one model feeds another, ensure downstream models aren’t bottlenecked.

What they must present

- A comparative table (e.g., YOLO-Nano vs. YOLO-Small vs. BlazePose)
- Measured latency on their device or capped PC
- A clear explanation of why the chosen model meets the constraints

Scheduling the Components

On an edge system, scheduling is as important as model accuracy. Some examples of Scheduling Considerations:

- **Temporal allocation**
  - Run heavy models (e.g., full pose estimation) at lower FPS (e.g., 5–10 FPS)
  - Run lightweight trackers between inference frames (e.g., 30 FPS)
- **Parallelism**
  - Use separate threads/cores:
    - Thread 1: Face detection
    - Thread 2: Keypoint inference
    - Thread 3: Fusion + decision logic
  - Ensure threads match the CPU/NPU capabilities of the device
- **Priority handling**
  - Safety/feedback loops get real-time priority
  - Secondary analytics run opportunistically
- **Graceful degradation**
  - If CPU load spikes, drop resolution or skip frames safely

Scheduling the Components

Deliverables students must show:

- A pipeline diagram with clear dependencies
- A scheduler timeline showing when each component runs
- Evidence of profiling showing CPU/GPU/NPU utilization

Students should demonstrate:

- A modular design,
- A well-justified model choice,
- A realistic scheduling strategy,
- And measurements validating that the pipeline works under edge constraints (or PC caps when hardware is unavailable).

These components make up a strong, credible, and well-assessed INF2009 project.

What We Assess

We look for **evidence against the rubric dimensions** used in your briefings:

- **Scope Coverage (esp. non-functional)**
  - Have you addressed latency, bandwidth, privacy, availability, and power explicitly?
- **Integration Quality**
  - Sensor → compute → storage/comm → actuation flows are coherent and reliable.
- **User Experience**
  - UI/telemetry supports deployment, troubleshooting, and safe operation.
- **Attention to Detail**
  - Responsiveness, consistency, and overall polish in the deliverables and demo.

Thank you

Email: ernest.tan@singaporetech.edu.sg